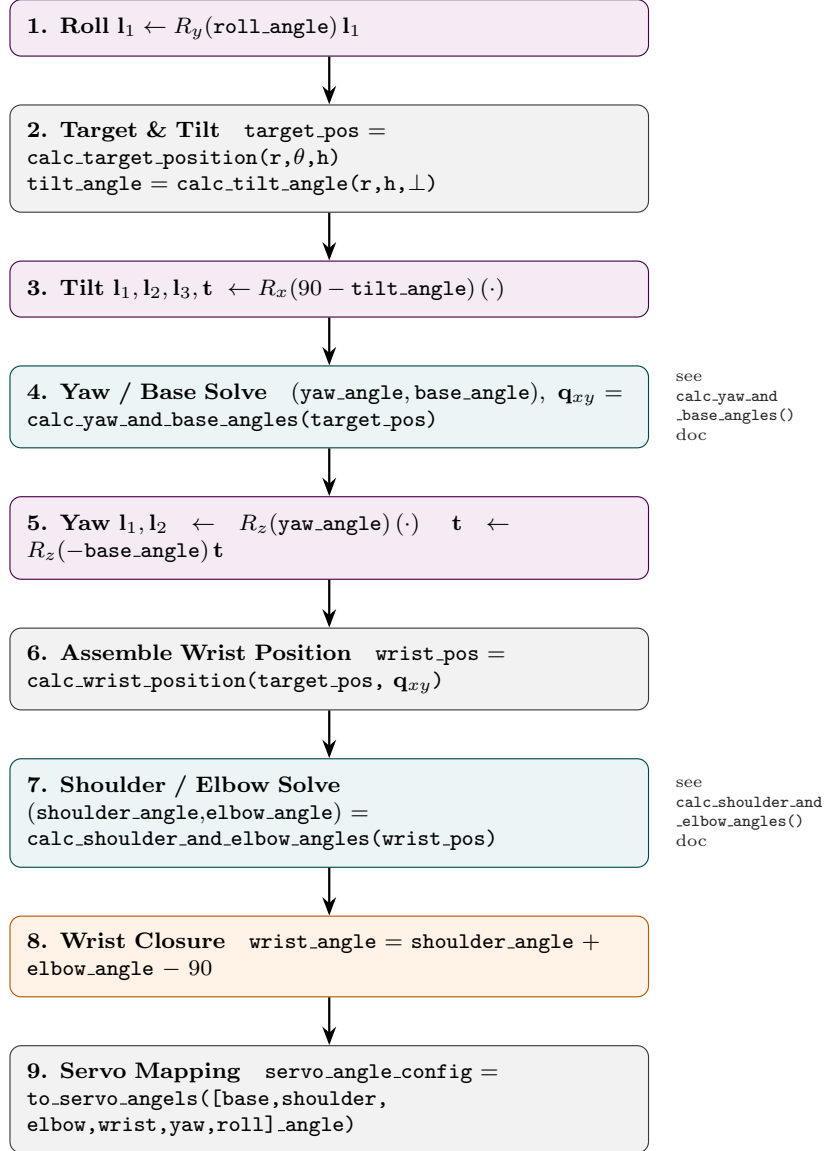


IKCompiler.calc_angle_config()



Violet: rotates one or more of the four translation vectors $\mathbf{l}_1, \mathbf{l}_2, \mathbf{l}_3, \mathbf{t}$ in place. **Teal:** delegates to an already-documented solve. **Gray:** pure assembly/bookkeeping. **Orange:** a closed-form step derived here.

1 Setup

$$\mathbf{l}_1 = (0, 0, L_1), \quad \mathbf{l}_2 = (L_2, 0, 0), \quad \mathbf{l}_3 = (0, 0, L_3), \quad \mathbf{t} = (0, 0, \text{wrist_arm_length})$$

are **self** attributes – they persist and are mutated *in place* across the whole method. **roll_angle** and **wrist_perpendicular_2_ground** ($\perp$ above) are free inputs: **roll_angle** is never solved for, only used to orient $\mathbf{l}_1$ and passed straight through to the output.

2 Transform Bookkeeping

Steps 1, 3, 5 apply a different subset of {roll, tilt, yaw/base} to each vector:

vector	roll (y)	tilt ($x, 90 - \text{tilt_angle}$)	yaw/base (z)
$\mathbf{l}_1$	✓	✓	✓ yaw_angle
$\mathbf{l}_2$	–	✓	✓ yaw_angle
$\mathbf{l}_3$	–	✓	–
$\mathbf{t}$	–	✓	✓ $-\text{base_angle}$

This asymmetry looks worse than it is. A z -axis rotation leaves the z component fixed:

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \implies (R_z(\theta) \mathbf{v})_z = v_z \quad (1)$$

and `calc_wrist_position` (§3) only ever reads $(\mathbf{l}_1 + \mathbf{l}_2 + \mathbf{l}_3)_z$ – never their x, y . So step 5’s yaw rotation of $\mathbf{l}_1, \mathbf{l}_2$, and $\mathbf{l}_3$ never receiving one, make *no difference to this call’s return value*: the z -sum is identical either way. They do still overwrite `self.l1_translation_vec` / `self.l2_translation_vec` with new x, y – state that only matters if something later reads those vectors’ full 3D value (nothing currently does). $\mathbf{t}$ is the one vector whose full vector (not just z) is used, so its yaw/base rotation in step 5 is the one that actually matters for the result.

3 Assembling the Wrist Position

$$\text{yaw_position}_z = (\text{target_pos} + \mathbf{l}_1 + \mathbf{l}_2 + \mathbf{l}_3)_z \quad (2)$$

$$\text{yaw_position} = (q_x, q_y, \text{yaw_position}_z) \quad (3)$$

$$\text{wrist_pos} = \text{yaw_position} + \mathbf{t} \quad (4)$$

q_x, q_y come from the circle-tangent solve in `calc_yaw_and_base_angles` – this step is pure vector addition, no new geometry is solved here.

4 Wrist Closure

Let ϕ be the world elevation (angle from horizontal) of segment $E\mathbf{w}$ – the elbow-to-wrist link, continuing the notation of the shoulder/elbow doc. At full extension (`elbow_angle` = 180°) ϕ equals `shoulder_angle`; bending away from that by $(180^\circ - \text{elbow_angle})$ gives, in general,

$$\phi = \text{shoulder_angle} + \text{elbow_angle} - 180^\circ \quad (5)$$

`wrist_angle` follows the same “ 90° = neutral” servo convention used throughout (`calc_tilt_angle, to_servo_angels`):

$$\text{wrist_angle} = \phi + 90^\circ = \text{shoulder_angle} + \text{elbow_angle} - 90^\circ \quad (6)$$

This assumes the shoulder/elbow solve reproduces the same orientation the earlier vector rotations (steps 1–5) already committed $\mathbf{t}$ to – i.e. the two halves of the method are only consistent because `tilt_angle` and the shoulder/elbow triangle are both driven by the same `target_pos`.

5 Notation $\leftrightarrow$ Code

Symbol	Variable
$\mathbf{l}_1, \mathbf{l}_2, \mathbf{l}_3$	<code>l1_translation_vec, l2_translation_vec, l3_translation_vec</code>
$\mathbf{t}$	<code>tilt_translation_vec</code>
$\mathbf{q}_{xy}$	<code>yaw_pos_xy</code>
ϕ	(elevation of segment $E\mathbf{w}$, not itself a variable)